Knightsec

University of Central Florida

(Team 11)

Intrusion Report

Table of Contents

Introduction 3

Active Directory 3

Admin User Interface 3

Management Backend 5

Public Website (WWW) 10

Ticketing Machine 16

Introduction

This report is meant to list the detected intrusions for each machine. This report is structured where each section is named after a machine that contains numbered findings. Each finding will contain info on malicious activity we found while securing the network and malicious activity team 11 detected when the network started being attacked by malicious actors starting at 7:30AM EST. Finding may or may not include screenshots or other detailed evidence of an intrusion.

Active Directory

External IP Address	Internal IP Address	Hostname
76.5.61.10	10.1.0.6	ad.team11.isucdc.com

Finding 1: Overpermissioned Active Directory Users

- Regular users that are supposed to be only in the “Citizens” group in the domain were, by default, configured in the “Domain Admins” group.

Admin User Interface

External IP Address	Internal IP Address	Hostname
76.5.61.40	10.1.0.4	admin.team11.isucdc.com

Finding 1: Flask Misconfiguration Can Lead to RCE

- In the source code of the admin-ui application, our team found that on starting the application, it is started with the evalex flag set to True. This is a very bad security misconfiguration, since this means that whenever an

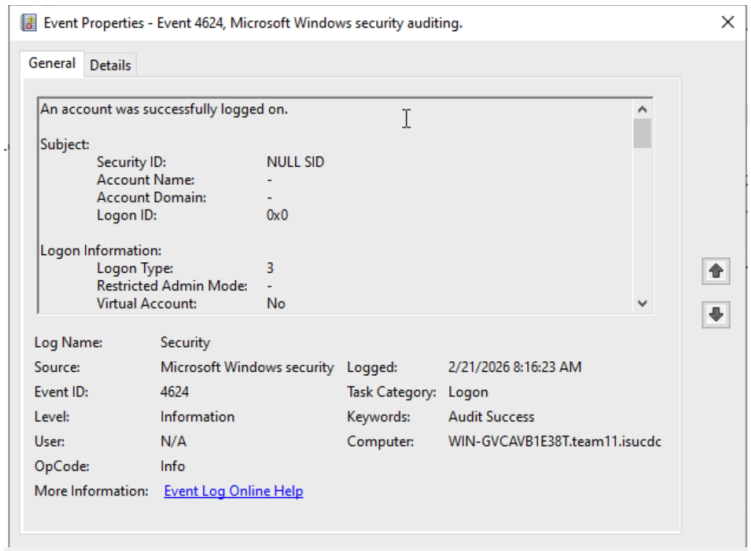
exception is triggered on the web application, an interactive terminal opens up that allows any user to execute arbitrary Python commands. This feature is meant only to be used for debugging and testing purposes, and should not be enabled on a production deployment

```python
import os

from flask import Flask
from werkzeug.debug import DebuggedApplication

from app import routes
from app.config import Config


def create_app():
    app = Flask(__name__)

    # Load configuration
    app.config.from_object(Config)

    app.wsgi_app = DebuggedApplication(app.wsgi_app, evalex=True)
    app.wsgi_app.trusted_hosts = {"localhost", "127.0.0.1", ".isucdc.com"}



    app.register_blueprint(routes.bp)

    return app
```

- All that needs to be done to mitigate this is to set evalex to False.

Finding 2: Attempted login during attack

- We detected a malicious login to the Admin UI box at around 9:33 AM EST. After detecting this login we changed credentials to the local administrator account. The image below shows some details from this attempt.



Management Backend

External IP Address	Internal IP Address	Hostname
76.5.61.30	10.1.0.2	mgmt.team11.isucdc.com

Finding 1: Mitigating PAM Backdoor

- To ensure that PAM (Pluggable Authentication Module) hasn't been backdoored (which is a common tactic for red team), PAM was reinstalled using `sudo dnf reinstall pam`.

Finding 2: Overpermissioned Directories

- Looking at the directories that hold the application's source code, our team found that the directories' permissions were too open, allowing regular users that shouldn't have access to the application's source code to view and

modify it.

```
[cdc@mgmt ~]$ ls -la
total 20
drwx------. 4 cdc  cdc  138 Feb  1 12:49 .
drwxr-xr-x. 5 root root  92 Oct  9  2021 ..
-rw-------  1 cdc  cdc  176 Feb 10 15:47 .bash_history
-rw-r--r--. 1 cdc  cdc   18 Apr 21  2021 .bash_logout
-rw-r--r--. 1 cdc  cdc  141 Apr 21  2021 .bash_profile
-rw-r--r--. 1 cdc  cdc  376 Apr 21  2021 .bashrc
drwx------  3 cdc  cdc   17 Jan 11 12:22 .cache
drwxrwxrwx  5 cdc  cdc   85 Feb  8 12:39 icdc26-backend-mgmt
-rw-r--r--. 1 cdc  cdc  658 Apr  7  2020 .zshrc
[cdc@mgmt ~]$ ls -la icdc26-backend-mgmt/
total 12
drwxrwxrwx  5 cdc  cdc    85 Feb  8 12:39 .
drwx------. 4 cdc  cdc   138 Feb  1 12:49 ..
drwxrwxrwx  5 cdc  cdc   190 Jan 21 16:25 app
drwxr-xr-x  2 root root   36 Feb  8 12:42 envvars
-rw-rw-rw-  1 cdc  cdc  7759 Jan 13 13:05 README.md
-rw-rw-rw-  1 cdc  cdc   175 Jan 11 17:10 requirements.txt
drwxrwxr-x  5 cdc  cdc   100 Jan 11 12:22 venv
[cdc@mgmt ~]$ _
```

- To mitigate this, our team promptly modified the permissions so that only the administrators can view and modify the application's source code.

Finding 3: Weak Secret Used for Session Token Signature

- In the application's source code, a weak secret key was being used for the signing of the application's session tokens.
- Since the secret key was "cdc", an attacker could trivially brute force this key and use it to create valid sessions with any user as long as they have the user's username.

```
❯ rg -i "= \"cdc\""
mgmt/icdc26-backend-mgmt/app/config.py
9:    SECRET_KEY = "cdc"
20:    LDAP_PASS = "cdc"  # LDAP bind password

www/icdc2026frontend-main/app/config.py
9:    SECRET_KEY = "cdc"
```

- To mitigate this, we changed the secret key of the application to a much longer key with more complexity and one that would be virtually impossible to brute force.

Finding 4: Flask Misconfiguration Can Lead to RCE

- In the source code of the management application, our team found that on starting the application, it is started with the evalex flag set to True. This is a very bad security misconfiguration, since this means that whenever an exception is triggered on the web application, an interactive terminal opens up that allows any user to execute arbitrary Python commands. This feature is meant only to be used for debugging and testing purposes, and should not be enabled on a production deployment.

```python
def create_app():
    app = Flask(__name__)

    CORS(app, origins=["*"])

    # Load configuration
    app.config.from_object(Config)

    app.wsgi_app = DebuggedApplication(app.wsgi_app, evalex=True)
    app.wsgi_app.trusted_hosts = {"localhost", "127.0.0.1", ".isucdc.com"}
```

- All that needs to be done to mitigate this is to set the evalex flag to False.

Finding 5: Excessive Privilege Assignment

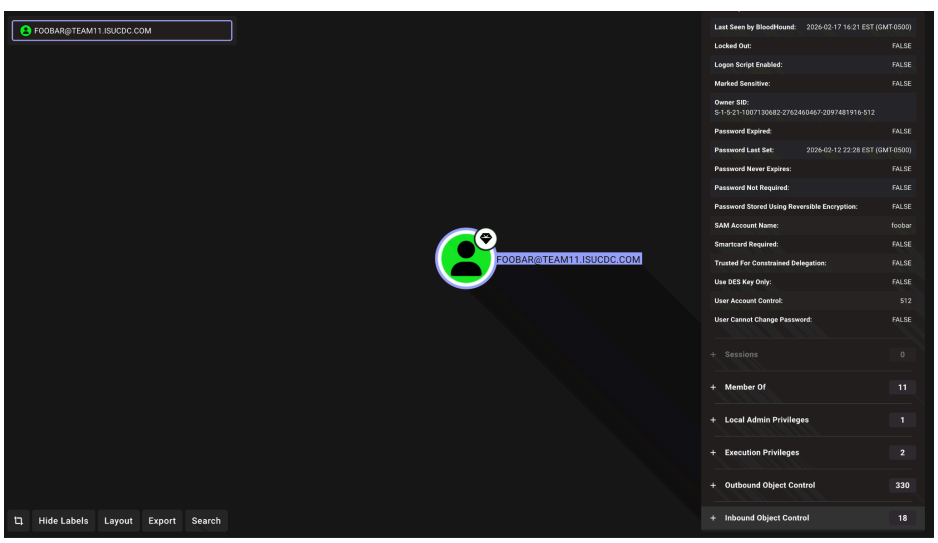
- Before the patch, whenever a user registered an account on the website, since all machines are domain joined, their account was added to the Active Directory domain. However, when their accounts were created, they get added to the “Transit Admins” group which essentially has the same privilege as a Domain Administrator.

```python
def create_ldap_user(username, password):
    conn = get_ldap_connection()
    user_attributes = {
        "objectClass": ["top", "person", "organizationalPerson", "user"],
        "cn": username,
        "sAMAccountName": username,
        "userPrincipalName": f"{username}@{Config.LDAP_DOMAIN}",
        "displayName": username,
    }
    # Create the user
    conn.add(f"cn={username}," + Config.LDAP_DN, attributes=user_attributes)
    # Set the password
    ad_modify_password(conn, f"cn={username}," + Config.LDAP_DN, password, None)
    # Enable the user
    conn.modify(
        f"cn={username}," + Config.LDAP_DN,
        {"userAccountControl": [(MODIFY_REPLACE, [512])]},
    )

    add_to_group(username, Config.ADMIN_GROUP)

    conn.unbind()
    return conn.result["description"] == "success"
```

- As seen in the code, the ‘add_to_group()’ function specifies the group to add to as ‘Config.ADMIN_GROUP’, which is a very serious security misconfiguration, allowing anyone to become a Domain Administrator.
- In order to mitigate this, the source code was edited to change the 2nd argument of this function call to “Citizens”, as this should be the group that users are added to when they register an account.

```python
def create_ldap_user(username, password):
    conn = get_ldap_connection()
    user_attributes = {
        "objectClass": ["top", "person", "organizationalPerson", "user"],
        "cn": username,
        "sAMAccountName": username,
        "userPrincipalName": f"{username}@{Config.LDAP_DOMAIN}",
        "displayName": username,
    }
    # Create the user
    conn.add(f"cn={username}," + Config.LDAP_DN, attributes=user_attributes)
    # Set the password
    ad_modify_password(conn, f"cn={username}," + Config.LDAP_DN, password, None)
    # Enable the user
    conn.modify(
        f"cn={username}," + Config.LDAP_DN,
        {"userAccountControl": [(MODIFY_REPLACE, [512])]},
    )

    add_to_group(username, "Citizens")
```

- Below is a Bloodhound ingest displaying a user account that was registered before the patch. As seen, the user has “Outbound Object Control” on every object in the domain and has the diamond on its user, meaning the user is a Domain Administrator.



Public Website (WWW)

External IP Address	Internal IP Address	Hostname
76.5.61.50	N/A	www.team11.isucdc.com

Finding 1: Mitigating PAM Backdoor

- To ensure that PAM (Pluggable Authentication Module) hasn't been backdoored (which is a common tactic for red team), PAM was reinstalled using sudo dnf reinstall pam.

Finding 2: Netcat Backdoor

- Upon checking for listening sockets and running processes, four persistent netcat listeners were found to be running on this machine.
- This allows an attacker to establish a shell on the machine unauthenticated, allowing for a full compromise of the server.

```
[root@www ~]# ss -tulpn
Netid State  Recv-Q  Send-Q   Local Address:Port    Peer Address:Port Process
udp   UNCONN 0       0              0.0.0.0:111          0.0.0.0:*     users:(("rpcbind",pid=907,fd=5),("systemd",pid=1,fd=35))
udp   UNCONN 0       0            127.0.0.1:323          0.0.0.0:*     users:(("chronyd",pid=961,fd=5))
udp   UNCONN 0       0                 [::]:111             [::]:*     users:(("rpcbind",pid=907,fd=7),("systemd",pid=1,fd=37))
udp   UNCONN 0       0                [::1]:323             [::]:*     users:(("chronyd",pid=961,fd=6))
tcp   LISTEN 0       5            127.0.0.1:44321        0.0.0.0:*     users:(("pmcd",pid=1186,fd=0))
tcp   LISTEN 0       128            0.0.0.0:22           0.0.0.0:*     users:(("sshd",pid=978,fd=3))
tcp   LISTEN 0       10             0.0.0.0:12345        0.0.0.0:*     users:(("nc",pid=951,fd=3))
tcp   LISTEN 0       128            0.0.0.0:80           0.0.0.0:*     users:(("flask",pid=974,fd=3))
tcp   LISTEN 0       4096           0.0.0.0:111          0.0.0.0:*     users:(("rpcbind",pid=907,fd=4),("systemd",pid=1,fd=34))
tcp   LISTEN 0       10             0.0.0.0:1337         0.0.0.0:*     users:(("nc",pid=956,fd=3))
tcp   LISTEN 0       5            127.0.0.1:4330         0.0.0.0:*     users:(("pmlogger",pid=2317,fd=9))
tcp   LISTEN 0       10             0.0.0.0:12069        0.0.0.0:*     users:(("nc",pid=953,fd=3))
tcp   LISTEN 0       5                [::1]:4330             [::]:*     users:(("pmlogger",pid=2317,fd=10))
tcp   LISTEN 0       128                [::]:22              [::]:*     users:(("sshd",pid=978,fd=5))
tcp   LISTEN 0       4096               [::]:111             [::]:*     users:(("rpcbind",pid=907,fd=6),("systemd",pid=1,fd=36))
tcp   LISTEN 0       5                [::1]:44321            [::]:*     users:(("pmcd",pid=1186,fd=3))
[root@www ~]#
```

```
949  0.0  0.0   7136  3528 ?        Ss   Feb08   0:00 bash /usr/bin/ticketmon
951  0.0  0.1  12264  5112 ?        S    Feb08   0:00  \_ nc -kl 0.0.0.0 12345
952  0.0  0.0   7136  3504 ?        S    Feb08   0:00  \_ bash
953  0.0  0.1  12264  5196 ?        S    Feb08   0:00  \_ nc -kl 0.0.0.0 12069
955  0.0  0.0   7136  3516 ?        S    Feb08   0:00  \_ bash
956  0.0  0.1  12264  5152 ?        S    Feb08   0:00  \_ nc -kl 0.0.0.0 1337
957  0.0  0.0   7136  3524 ?        S    Feb08   0:00  \_ bash
```

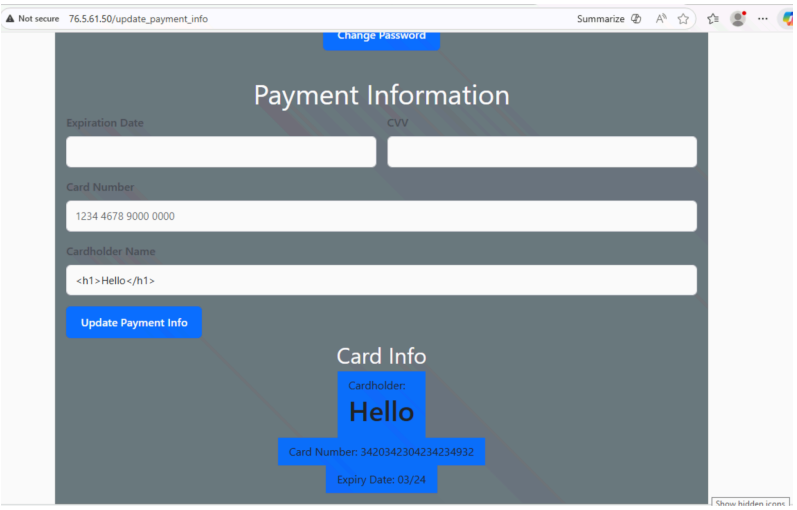
- After further investigation, our team found that these listeners were started by a malicious bash script that was being run by a malicious backdoored service on the machine, titled "ticket_mon.service".

```
  ticket_mon.service        loaded active running "TicketServ-rs"
  tuned.service             loaded active running Dynamic System Tuning Daemon
  user@1000.service         loaded active running User Manager for UID 1000

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.
32 loaded units listed.
[root@www cdc]# systemctl status ticket_mon.service
● ticket_mon.service - "TicketServ-rs"
     Loaded: loaded (/etc/systemd/system/ticket_mon.service; enabled; preset: disabled)
     Active: active (running) since Tue 2026-02-10 17:41:10 EST; 1 day 18h ago
   Main PID: 946 (bash)
      Tasks: 7 (limit: 23073)
     Memory: 5.1M (peak: 5.3M)
        CPU: 59ms
     CGroup: /system.slice/ticket_mon.service
             ├─946 bash /usr/bin/ticketmon
             ├─948 nc -kl 0.0.0.0 12345
             ├─949 bash
             ├─950 nc -kl 0.0.0.0 12069
             ├─951 bash
             ├─952 nc -kl 0.0.0.0 1337
             └─953 bash

Feb 10 17:41:10 www.team11.isucdc.com systemd[1]: Started "TicketServ-rs".
```

- The service was set to “enabled”, meaning it would start up after every system reboot.
- To prevent this from causing any issues, our team quickly disabled the service, deleted it, and successfully removed it from the system.

Finding 3: HTML Injection

- When inputting cardholder information in the page’s portal, user input is not validated safely, allowing a user to input arbitrary HTML that is directly injected into the page’s HTML code.



- A malicious actor could leverage this to perform various malicious actions, such as phishing a user for account takeover via exfiltrating session cookies, redirecting a user to a malicious website, and fully changing the appearance of the webpage.
- In order to mitigate this, user input sanitization could be used in the backend logic, and also CSP (Content Security Policy), which prevents certain JavaScript content from being executed on the page.

Finding 4: Weak Secret Used for Session Token Signature

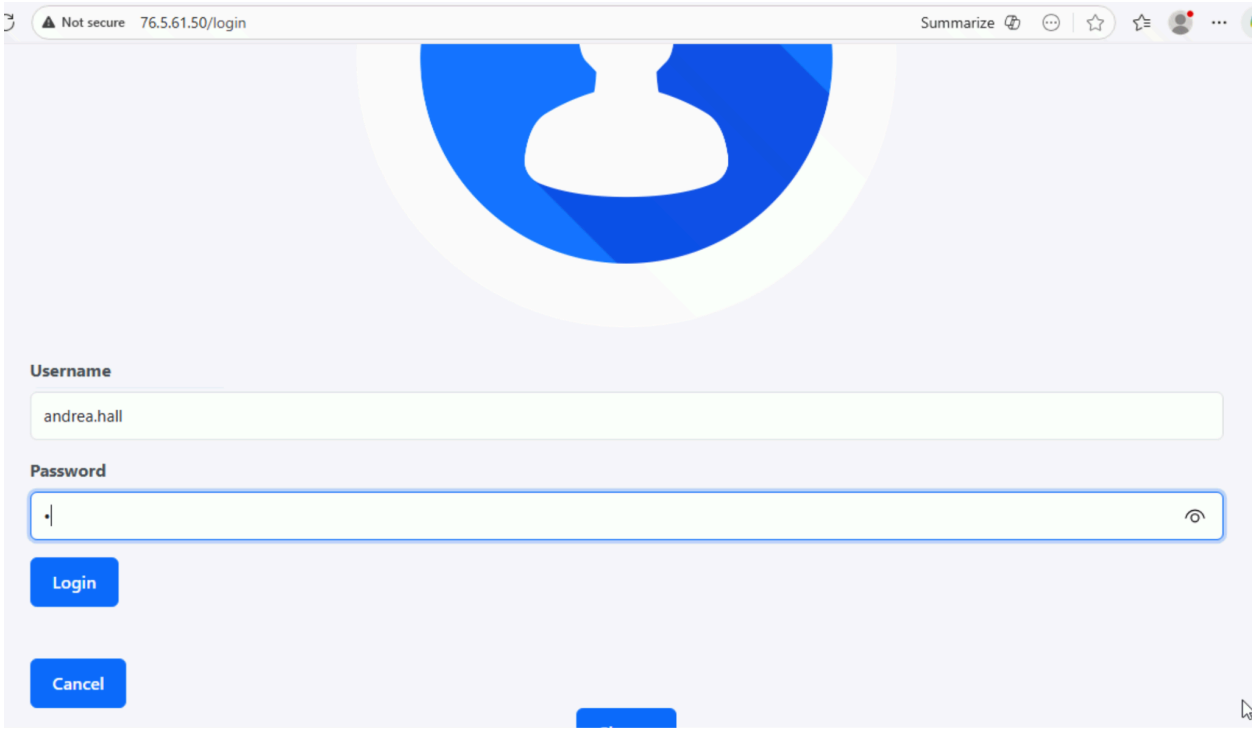
- In the application's source code, a weak secret key was being used for the signing of the application's session tokens.
- Since the secret key was "cdc", an attacker could trivially brute force this key and use it to create valid sessions with any user as long as they have the user's username.

```
❯ rg -i "= \"cdc\""
mgmt/icdc26-backend-mgmt/app/config.py
9:    SECRET_KEY = "cdc"
20:    LDAP_PASS = "cdc"  # LDAP bind password

www/icdc2026frontend-main/app/config.py
9:    SECRET_KEY = "cdc"
```

- To mitigate this, we changed the secret key of the application to a much longer key with more complexity and one that would be virtually impossible to brute force.

Finding 5: Broken Authentication

- On the main website, any user can enter a valid username such as “Administrator” and can enter a random password and be authenticated to that account.



- This is a very severe security misconfiguration, especially since these machines are all domain-joined. The domain administrator user “Administrator” can be logged into by any user, allowing for a full domain compromise.
- Below is the vulnerable line of code. The flaw in logic was that the backend code didn’t check for a password for the user, but only checked for if the user existed in the Active Directory domain, and authenticated the user if that were the case.

```python
@bp.route("/login", methods=["POST"])
def login():
    data = request.get_json()
    username = data.get("username")
    password = data.get("password")

    if not username or not password:
        return jsonify({"error": "Username and password required"}), 400

    # success, is_admin = authenticate(username, password)
    success = ldap_exists(username)
    is_admin = True

    if not success:
        return jsonify({"error": "Authentication failed"}), 401

    session["username"] = username
    session["is_admin"] = is_admin

    return (
        jsonify({"message": "Login successful"}),
        200,
    )
```

- Upon seeing this, our team quickly patched this by using the appropriate function used for authentication (authenticate()), and upon testing our patch, it worked with no issues.

```python
    return jsonify({"error": "Username and password required"}), 400

    # The below line is the fix for authentication bypass
    success, is_admin = authenticate(username, password)
    # success = ldap_exists(username)

    # Added the following check to ensure only the "Administrator" user is given admin access to the page
    if username == "Administrator" and success:
        is_admin = True
    else:
        is_admin = False
```

Finding 6: IP tables flushed during attack

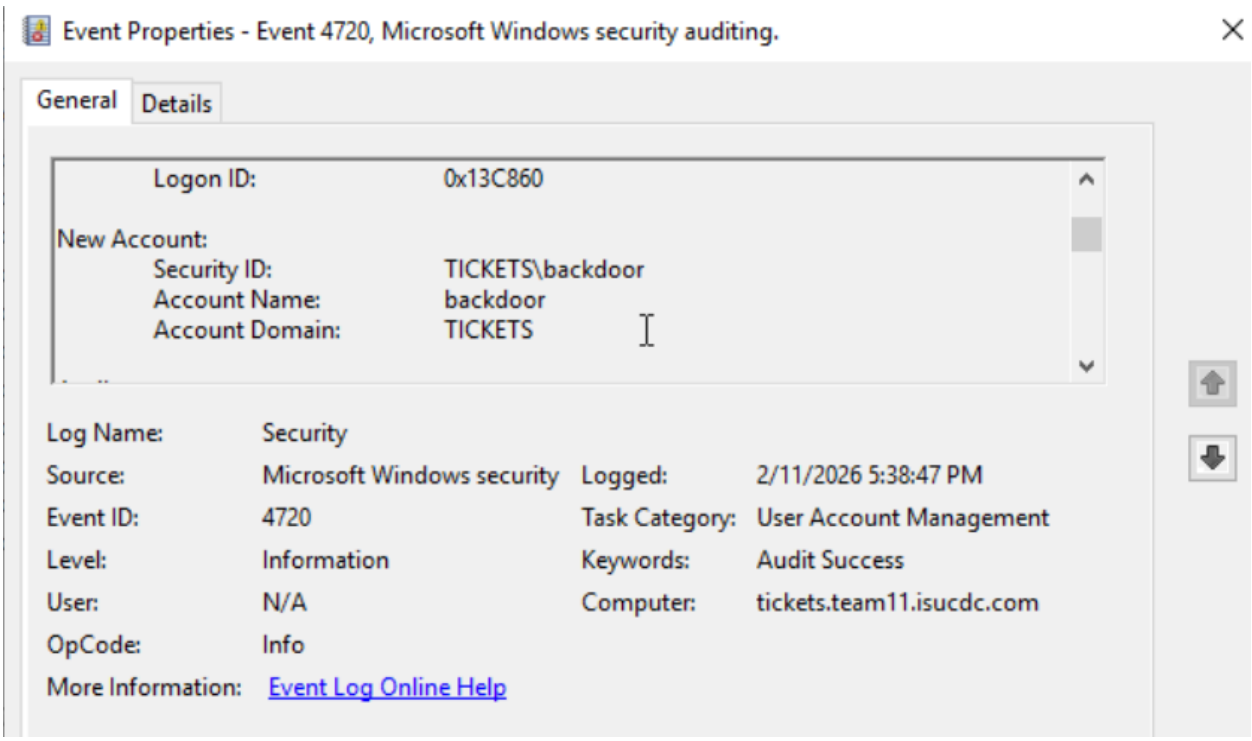
During the attack at roughly around 8:30 AM EST our team noticed our IPtable firewall rules were flushed. To mitigate this in the future we cycled credentials for the local admin, threat hunted the box, and restored our

IPtable firewalls we created from a backup. We do not have evidence of this finding as of now.

Ticketing Machine

External IP Address	Internal IP Address	Hostname
76.5.61.20	10.1.0.3	tickets.team11.isucdc.com

Finding 1: Backdoored User



- Upon doing auditing of the local users of the ticketing machine a “backdoor” user was found.